ELEC 475 Lab 2 Post-Lab Report

ELEC 475 — Lab 2 Post-Lab Report

Name: Nicole Wu · **Student number:** 20381046 · **Partner:** None

GitHub repository: https://github.com/nic04wjt/elec475-lab

Batch Normalization gave the best AlexNet-64 on CIFAR-10 (79.93% validation accuracy after 10 epochs, vs 75.14% with LRN and 73.52% with no normalization). Data augmentation then raised it to 82.83% and cut the generalization gap from 11.72 to 2.99 points. On the held-out Tiny ImageNet-200 validation set the same network reached 30.15%, about 60 times the 0.5% chance level.

1. Architectural Specifications

All four networks take a 3×64×64 RGB image and end in a linear layer with one output per class (10 for CIFAR-10, 200 for Tiny ImageNet). Layer sizes follow the formulas below.

- **Output size:** $N_{out} = \lfloor (N_{in} - K + 2P) / S \rfloor + 1$
- **Receptive field:** $r_l = r_{(l-1)} + (K_l - 1) \times j_{(l-1)}$
- **Cumulative stride (jump):** $j_l = j_{(l-1)} \times S_l$

where N is the spatial size, K the kernel size, P the padding and S the stride of each layer.

The second formula gives the receptive field r and the cumulative stride (jump) j of each layer, starting from r = j = 1 at the input.

1.1 Base LeNet-5

Layer	Configuration	Output shape	Params
Conv1 + Tanh	6 filters, 5×5, s=1, p=0	6×60×60	456
AvgPool1	2×2, s=2	6×30×30	0
Conv2 + Tanh	16 filters, 5×5, s=1, p=0	16×26×26	2,416
AvgPool2	2×2, s=2	16×13×13	0
Flatten	16×13×13	2,704	0
FC1 + Tanh	2,704 → 120	120	324,600

Layer	Configuration	Output shape	Params
FC2 + Tanh	120 → 84	84	10,164
FC3	84 → 10	10	850

Total 338,486 parameters: 2,872 in the convolutions and 335,614 (99.2%) in the dense head. The final receptive field is 16×16 pixels, a quarter of the image width.

1.2 Modernized LeNet

Two changes from LeNet-5: Tanh → ReLU, and 2×2 average pooling → 3×3 overlapping max pooling with stride 2. Padding of 2 keeps each convolution's output the same size as its input.

Layer	Configuration	Output shape	Params
Conv1 + ReLU	6 filters, 5×5, s=1, p=2	6×64×64	456
MaxPool1	3×3, s=2	6×31×31	0
Conv2 + ReLU	16 filters, 5×5, s=1, p=2	16×31×31	2,416
MaxPool2	3×3, s=2	16×15×15	0
Flatten	16×15×15	3,600	0
FC1 + ReLU	3,600 → 120	120	432,120
FC2 + ReLU	120 → 84	84	10,164
FC3	84 → 10	10	850

Total 446,006 parameters. The convolutions are unchanged; the 107,520 extra parameters all come from FC1, because the larger 15×15 map flattens to 3,600 features instead of 2,704. The receptive field grows to 19×19 pixels.

1.3 AlexNet-64 (from scratch)

Stage	Configuration	Output shape	Params	Receptive field
Conv1 + Norm + ReLU	64 filters, 5×5, s=2, p=2	64×32×32	4,864	5
MaxPool1	3×3, s=2	64×15×15	0	9
Conv2 + Norm + ReLU	192 filters, 5×5, s=1, p=2	192×15×15	307,392	25
MaxPool2	3×3, s=2	192×7×7	0	33
Conv3 + (BN) + ReLU	384 filters, 3×3, s=1, p=1	384×7×7	663,936	49
Conv4 + (BN) + ReLU	384 filters, 3×3, s=1, p=1	384×7×7	1,327,488	65
Conv5 + (BN) + ReLU	256 filters, 3×3, s=1, p=1	256×7×7	884,992	81
MaxPool3	3×3, s=2	256×3×3	0	97
Dropout + FC1 + ReLU	p=0.5, 2,304 → 1,024	1,024	2,360,320	—
Dropout + FC2 + ReLU	p=0.5, 1,024 → 1,024	1,024	1,049,600	—
FC3	1,024 → 10	10	10,250	—

"Norm" is set by `--norm`: `none` (Identity), `lrn` (LocalResponseNorm, size 5, α=1e-4, β=0.75, k=2) or `bn` (BatchNorm2d). As in the 2012 paper, LRN is applied only after Conv1 and Conv2; in `bn` mode BatchNorm follows all five convolutions. BatchNorm adds 2,560 learnable parameters (γ and β per channel); LRN adds none.

Total 6,611,402 parameters with BN (6,608,842 without): 3,188,672 (48%) in the convolutions and 3,420,170 (52%) in the dense head. This is higher than the manual's ≈6.3 M estimate, but it is exactly what the specified layers produce, and the same counting reproduces the manual's LeNet figures (338,486 and 446,006) exactly. By Pool3 the receptive field is 97 pixels, larger than the whole 64×64 input, so every final feature can see the entire image.

1.4 PyTorch Model Zoo AlexNet

`torchvision.models.alexnet` keeps its ImageNet design (built for 224×224 input), with only the last layer replaced by Linear(4096 → num_classes):

Stage	Configuration	Output on 64×64 input	Params
Conv1 + ReLU, MaxPool	64 filters, 11×11, s=4, p=2; pool 3×3 s=2	64×15×15 → 64×7×7	23,296
Conv2 + ReLU, MaxPool	192 filters, 5×5, p=2; pool 3×3 s=2	192×7×7 → 192×3×3	307,392
Conv3–5 + ReLU, MaxPool	384, 256, 256 filters, 3×3, p=1; pool 3×3 s=2	256×3×3 → 256×1×1	2,139,008
AdaptiveAvgPool2d	output 6×6	256×6×6 = 9,216	0
Dropout + FC 9,216 → 4,096 + ReLU		4,096	37,752,832
Dropout + FC 4,096 → 4,096 + ReLU		4,096	16,781,312
FC 4,096 → 10		10	40,970

Total 57,044,810 parameters, 95.7% of them in the dense head. On a 64×64 image the stride-4 first layer shrinks the feature map so quickly that it reaches 1×1 after the last pool. AdaptiveAvgPool2d then copies that single value into all 36 positions of a 6×6 grid, so FC1's 9,216 inputs hold only 256 distinct numbers. Most of FC1's 37.8 M weights therefore add no information on this input size.

2. Experimental Setup

Every run used Adam (learning rate 1e-3), cross-entropy loss, batch size 128 and global seed 42, with no early stopping. Accuracy and Macro F1 are the best validation epoch reported by `train.py`.

Runs	Hardware	Epochs
LeNet-5, Modernized LeNet (in-lab)	MacBook CPU	5

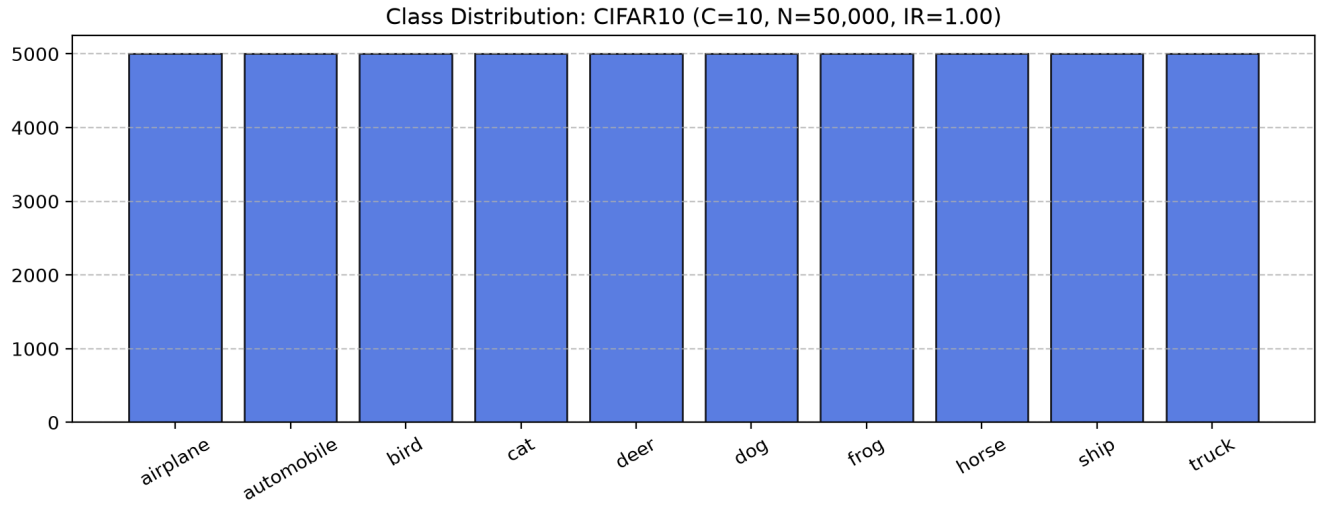
Runs	Hardware	Epochs
AlexNet-64 normalization study	Google Colab, NVIDIA Tesla T4	10
AlexNet-64 augmentation study	Google Colab, NVIDIA Tesla T4	15
AlexNet-64 on Tiny ImageNet	Google Colab, NVIDIA Tesla T4	10

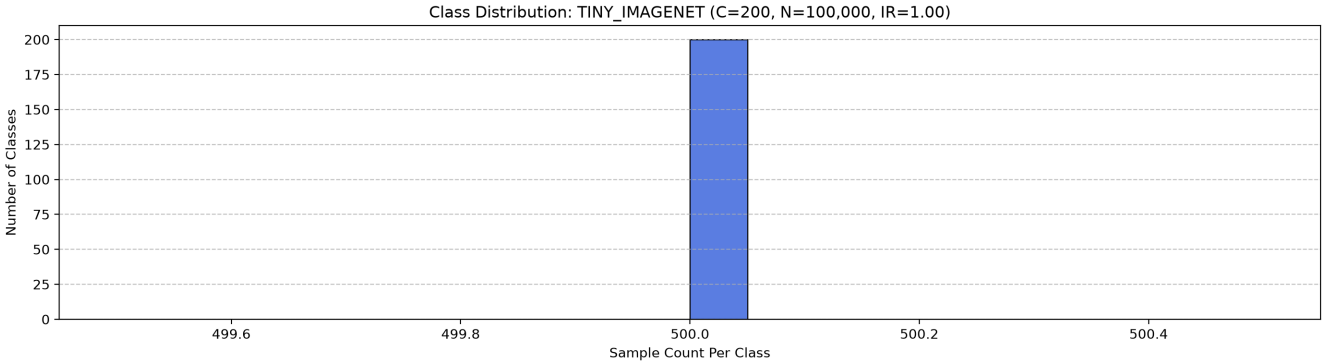
Notes that affect how the results should be read:

- **Timing is only comparable within one machine.** The LeNet epoch times were measured on a laptop CPU, so they cannot be compared directly with the AlexNet times on the T4 GPU. Inference latency for all models was measured on the T4 by `compare_models.py`.
- **The LeNet runs had no effective augmentation.** Their logs print `Augmentation: True`, but at that time `get_train_transforms()` still contained the template's `pass`, so only Resize, ToTensor and Normalize were applied. The pipeline was implemented before the final augmentation run.
- **Seeding is deterministic.** The first 10 epochs of the 15-epoch no-augmentation run match the 10-epoch BatchNorm run to every printed decimal, which confirms that `utils/seed.py` makes runs fully reproducible.
- **FLOPs are counted as 2 × multiply-accumulates** in the convolution and linear layers, as reported by the profiler in `train.py`. For the Zoo AlexNet, which was not trained, FLOPs were counted with PyTorch's `FlopCounterMode` using the same convention. The manual quotes ≈250 MFLOPs for AlexNet-64; the 437 MFLOPs measured here follow directly from the specified layer sizes (218.6 M MACs × 2).

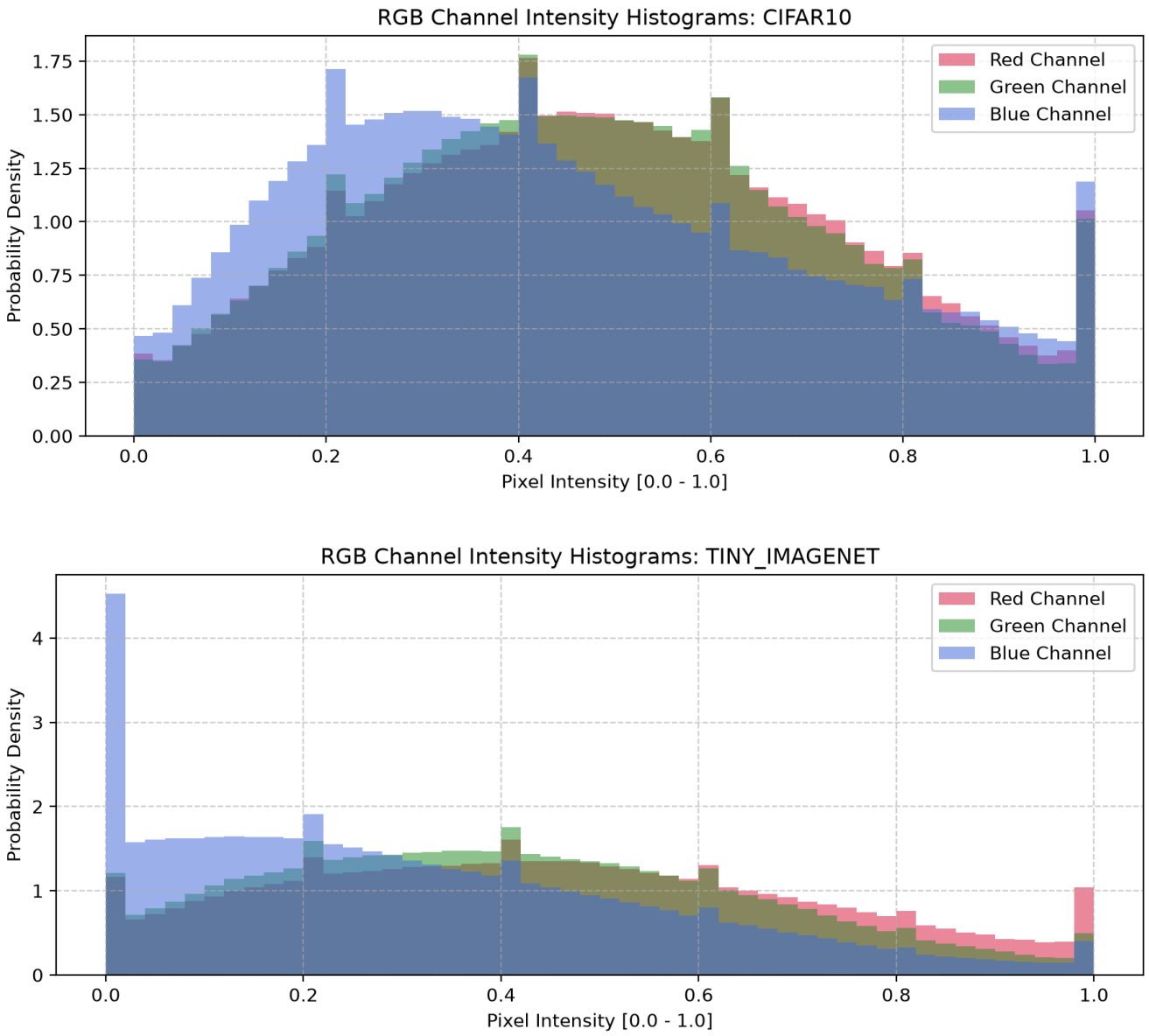
Exploratory data analysis (`utils/eda.py`)

Both datasets are perfectly class-balanced (imbalance ratio 1.00): CIFAR-10 has 5,000 training images in each of 10 classes, and Tiny ImageNet has 500 in each of 200. Because no class dominates, accuracy and Macro F1 track each other closely in every run, and Macro F1 needs no reweighting to be meaningful.

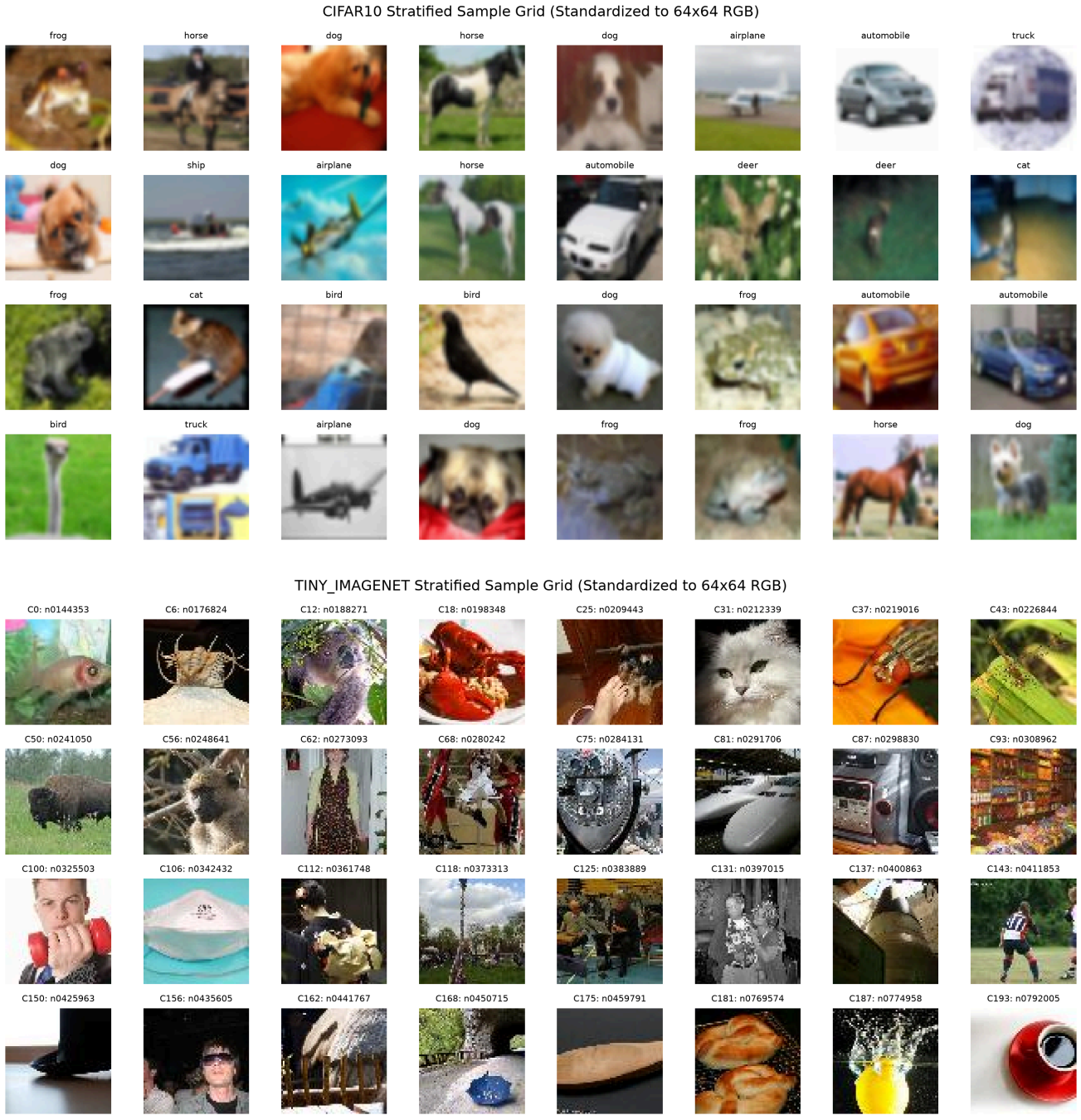




The channel histograms show why each dataset gets its own normalization statistics. In CIFAR-10 the blue channel sits darker than red and green (mean 0.447 vs 0.491 and 0.482), and all three channels spike at 1.0 from saturated white backgrounds. Tiny ImageNet is darker overall, with a large spike at 0 in the blue channel (dark and black regions) and a red spike at 1.0. Normalizing with each dataset's own per-channel mean and standard deviation centres the inputs for the first convolution.



The sample grids show the difference in image quality. CIFAR-10 images are visibly blurry, because they are upscaled from 32×32 to 64×64 and gain no new detail; objects are mostly centred. Tiny ImageNet images are sharp at their native 64×64 but show cluttered scenes, partial objects and people in many classes.



Transform pipeline diagnostic (`demo_transforms.py`)

```
[1] OpenCV (cv2) Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels]
(Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Color Space      : Converted BGR -> RGB
```

```
    - Memory Location  : Host System RAM (CPU)

[2] NumPy Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels]
(Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Memory Location  : Host System RAM (CPU)

[3] PyTorch (torchvision) Tensor Output:
    - Object Type      : <class 'torch.Tensor'>
    - Data Type (dtype): torch.float32
    - Shape (Layout)   : torch.Size([3, 64, 64])  --> [Channels, Height,
Width] (Channels-First)
    - Value Range      : Min = -1.9895, Max = 2.1265 (Standardized)
    - Memory Location  : cpu (Host CPU RAM)

Benchmark: Processing 1,000 Images (Resize + Flip + Normalize)
- OpenCV (CPU)            :    7.80 ms  (128,276 img/s) + Host-to-Device Copy
required
- NumPy (CPU)             :   20.34 ms  ( 49,173 img/s) + Host-to-Device Copy
required
- PyTorch (CPU Tensors)   :   87.92 ms  ( 11,374 img/s)
```

OpenCV and NumPy return channels-last float32 arrays scaled to [0, 1], while torchvision returns a channels-first [3, 64, 64] tensor that is already standardized with the dataset mean and standard deviation (range −1.99 to 2.13). Conv2d expects channels-first, so array outputs would need a permute and a copy to the GPU before training. OpenCV was the fastest at about 128,000 images/s, 11 times the torchvision CPU pipeline. This matches the training results: the torchvision augmentation pipeline running on Colab's 2 CPU cores roughly doubled the epoch time (Section 5).

3. Master Ablation Tables

Table 7.3a — Architecture and computational complexity

Model	Conv layers	Normalization / regularization	Augmentation	Params (M)	FLOPs (M)	Epoch time (s)	Latency (ms)
1. LeNet-5	2	None (Tanh, AvgPool)	None	0.338	7.17	86.3 (CPU)	0.35
2. Modern LeNet	2	None (ReLU, MaxPool)	None	0.446	9.19	89.3 (CPU)	0.38
3. AlexNet	5	Dropout 0.5	None	6.609	437.17	23.47	1.88
4. AlexNet + LRN	5	LRN, Dropout 0.5	None	6.609	437.17	25.37	2.07
5. AlexNet + BN	5	BN, Dropout 0.5	None	6.611	437.17	23.98	1.47
6. AlexNet + BN + Aug	5	BN, Dropout 0.5	Crop, flip, jitter	6.611	437.17	50.40	1.47
7. Zoo AlexNet	5	Dropout 0.5, AdaptiveAvgPool	None	57.045	188.18	not trained	1.93

Epoch times are means over all epochs of each run; the GPU rows are on the Tesla T4. Row 6 shares row 5's network, so its latency is the same model's.

Terminal output of `python compare_models.py --dataset cifar10` (Tesla T4):

```
================================================================================
ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark
Device        : cuda (Tesla T4)
Dataset       : CIFAR10 (10 classes)
Input Shape   : (1, 3, 64, 64)
================================================================================

| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency
(ms) | Throughput (img/s) |
| ----------------------------|------------|----------|-----------|----------
----|------------------- |
| 1. LeNet-5 (Base)           | 0.338      | 0.00     | 0.00      | 0.35
| 2853.6             |
| 2. LeNet-Modern (ReLU+Pool) | 0.446      | 0.00     | 0.00      | 0.38
| 2654.2             |
| 3. AlexNet-64 (None)        | 6.609      | 0.00     | 0.00      | 1.88
| 532.7              |
| 4. AlexNet-64 (LRN)         | 6.609      | 0.00     | 0.00      | 2.07
| 483.3              |
| 5. AlexNet-64 (BatchNorm)   | 6.611      | 0.00     | 0.00      | 1.47
| 681.4              |
| 6. Torchvision Zoo AlexNet  | 57.045     | 0.00     | 0.00      | 1.93
| 518.6              |
```

The harness's MACs and FLOPs columns printed 0.00 because its FLOP-counting library was not installed on Colab; installing `thop` instead crashed the GPU latency benchmark with a CPU/GPU device mismatch. The parameter, latency and throughput columns are valid. FLOPs in Table 7.3a therefore come from the profiler in `train.py`, which counts the same layers (2 × MACs), and from PyTorch's `FlopCounterMode` for the untrained Zoo AlexNet.

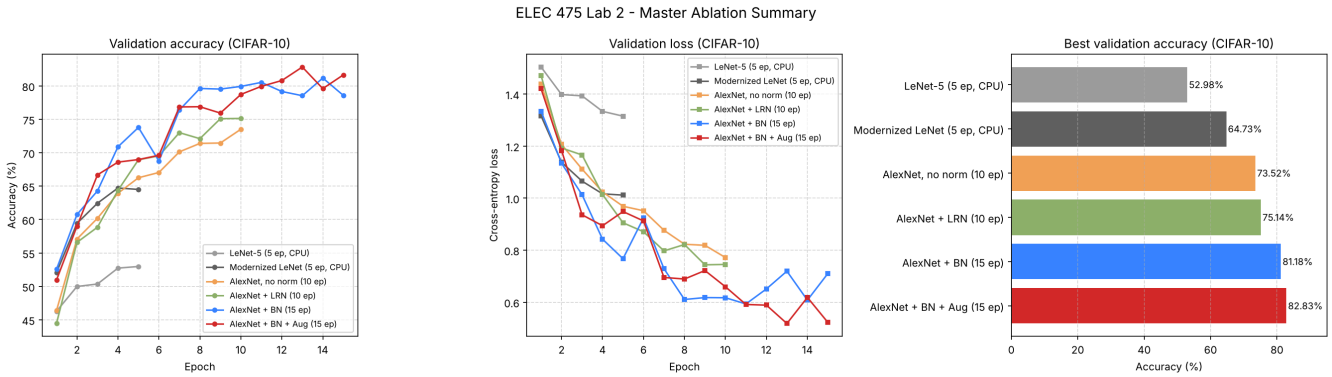
Table 7.3b — Predictive performance

Model	CIFAR-10 val acc (%)	CIFAR-10 val Macro F1 (%)	Tiny ImageNet val acc (%)	Tiny ImageNet val Macro F1 (%)
1. Base LeNet-5 (5 epochs)	52.98	53.12	not run	not run
2. Modernized LeNet (5 epochs)	64.73	64.44	not run	not run

Model	CIFAR-10 val acc (%)	CIFAR-10 val Macro F1 (%)	Tiny ImageNet val acc (%)	Tiny ImageNet val Macro F1 (%)
3. AlexNet (No Norm, 10 epochs)	73.52	73.48	not run	not run
4. AlexNet + LRN (10 epochs)	75.14	75.26	not run	not run
5. AlexNet + BatchNorm (10 epochs)	79.93	80.06	30.15	28.25
6. AlexNet + BN + Aug (15 epochs)	82.83	82.90	not run	not run
7. PyTorch Zoo AlexNet	not run	not run	not run	not run

For a fair comparison with row 6, AlexNet + BatchNorm trained for 15 epochs without augmentation reached 81.18% accuracy and 81.08% Macro F1. The Tiny ImageNet result in row 5 is AlexNet + BatchNorm without augmentation, measured on the official 10,000-image validation split after correcting the provided loader (see Section 6). Rows marked "not run" were not trained because of the Colab GPU time available before the deadline.

Ablation summary



The summary plots every CIFAR-10 run from its `train.py` epoch log. Each step up the ladder lifts the accuracy curve, and the augmented run is the only one whose validation loss is still falling at its last epoch.

Multi-seed evaluation (Mean ± Std)

AlexNet-64 + BatchNorm was trained three times on CIFAR-10 with identical settings (10 epochs, batch size 128, no augmentation), changing only the random seed. The standard deviation uses Bessel's correction (n − 1).

Seed	Best val acc (%)	Best val Macro F1 (%)	Mean epoch time (s)
42	79.93	80.06	23.98
1	78.12	77.72	22.79
2	80.02	80.02	22.94
Mean ± Std	**79.36 ± 1.07**	**79.27 ± 1.34**	23.24

Run-to-run variation from the seed alone is about ±1 point. This puts the other comparisons in context:

- **BatchNorm vs LRN and no normalization is robust.** BatchNorm's mean (79.36%) is 4.2 points above LRN (75.14%) and 5.8 points above no normalization (73.52%), about 4 standard deviations, so the normalization result is not a lucky seed.
- **LRN vs no normalization (1.6 points) is within noise.** That gap is about 1.5 standard deviations, so this study cannot say LRN helped.
- **The augmentation accuracy gain (1.65 points) is borderline.** It is about 1.5 standard deviations from a single seed each; the large drop in the generalization gap (11.72 → 2.99) is the stronger evidence for augmentation.

Extended ablation: batch size

BatchNorm normalizes with statistics computed over each mini-batch, so batch size directly controls how noisy those statistics are. AlexNet-64 + BatchNorm was trained with batch sizes 64 and 256, all other settings as in the seed-42 baseline (batch size 128, learning rate 1e-3, 10 epochs, no augmentation).

Batch size	Updates per epoch	Best val acc (%)	Δ acc vs 128 (pts)	Best val Macro F1 (%)	Δ F1 vs 128 (pts)	Final train acc (%)	Mean epoch time (s)
64	781	79.17	−0.76	79.17	−0.89	84.03	24.42
128 (baseline)	390	79.93	0.00	80.06	0.00	84.86	23.98

Batch size	Updates per epoch	Best val acc (%)	Δ acc vs 128 (pts)	Best val Macro F1 (%)	Δ F1 vs 128 (pts)	Final train acc (%)	Mean epoch time (s)
256	195	77.85	−2.08	77.66	−2.40	85.32	22.75

- **Batch size 64 performed the same as 128 within seed noise** (−0.76 points, less than one standard deviation). Its validation curve was the smoothest of the three, with no large dips, likely because twice as many updates per epoch let the BatchNorm running statistics track the weights more closely.
- **Batch size 256 lost 2.08 points**, about twice the seed standard deviation. With the learning rate fixed at 1e-3, it makes half as many updates per epoch as the baseline, so in 10 epochs it takes fewer optimization steps. It also showed the largest gap between training and validation accuracy (85.32% vs 77.85% best), consistent with large batches settling into sharper minima that generalize less well.
- **Epoch time changed little** (24.4 s to 22.8 s). Larger batches use the GPU more efficiently, but the T4 is already close to saturated at batch size 128, so the saving is only about 7%.

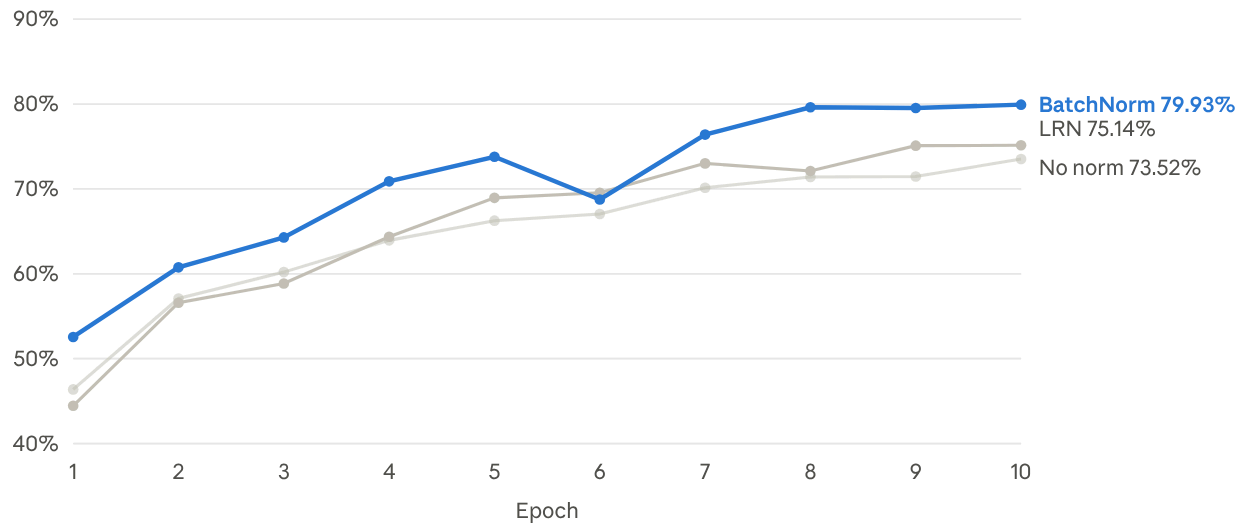
A larger batch would likely need a proportionally larger learning rate to keep up; with the default settings, batch size 128 is the best balance of accuracy and speed.

4. Normalization Study: None vs LRN vs BatchNorm2d

BatchNorm gave the highest validation accuracy and Macro F1, the lowest training loss at every epoch, and cost only 2% more time per epoch. LRN was slower than both and gained only 1.6 points over no normalization.

BatchNorm reaches 79.9% validation accuracy, 4.8 points ahead of LRN

AlexNet-64 on CIFAR-10, validation accuracy (%) by epoch, no augmentation, seed 42



train.py logs: alexnet_none.txt, alexnet_lrn.txt, alexnet_bn.txt · 10 epochs each, Tesla T4

Variant	Final train loss	Best val Macro F1 (%)	Mean epoch time (s)	Inference latency (ms)
No normalization	0.729	73.48	23.47	1.88
LRN (after Conv1, Conv2)	0.617	75.26	25.37	2.07
BatchNorm2d (after all 5 convs)	0.451	80.06	23.98	1.47

Did BatchNorm improve accuracy and Macro F1? Yes.

BatchNorm improved validation accuracy by 6.41 points over no normalization and 4.79 points over LRN; Macro F1 rose by 6.58 and 4.80 points. The advantage appears from the first epoch: training loss after epoch 1 was 1.48 with BatchNorm versus 1.80 without normalization and 1.76 with LRN.

BatchNorm normalizes each channel over the mini-batch and spatial dimensions (N, H, W), then applies a learned scale γ and shift β:

- **Normalize:** $\hat{x} = (x - \mu_B) / \sqrt{(\sigma^2_B + \varepsilon)}$
- **Scale and shift:** $y = \gamma \cdot \hat{x} + \beta$

where μ_B and σ^2_B are the mean and variance of the channel over the mini-batch, and ε is a small constant that avoids division by zero.

- **Stable layer inputs.** Each layer receives inputs with roughly zero mean and unit variance even as the weights before it change. This is the reduction of internal covariate shift described by Ioffe and Szegedy, so deeper layers stop chasing a moving input distribution.
- **A smoother loss landscape.** Normalization bounds how fast the loss and its gradient can change with the weights, so a learning rate of 1e-3 makes reliable progress instead of overshooting. This is why BatchNorm's training loss fell fastest across all 10 epochs.
- **Healthier ReLUs.** Centring pre-activations near zero keeps roughly half of each layer's ReLUs active, so fewer units die or saturate in the deep Conv3–Conv5 stack.
- **Mild regularization.** Batch statistics differ slightly from batch to batch, which adds noise similar to a weak form of dropout.

LRN divides each activation by the summed energy of its 5 neighbouring channels. With $k = 2$ and $\alpha = 1e\text{-}4$, the denominator stays close to $2^{0.75} \approx 1.68$ unless activations are very large, so LRN acts mostly as a fixed rescaling. Its 1.6-point gain over no normalization comes from a single seed and may be within run-to-run variation.

Did LRN or BatchNorm change the computational cost? Yes, slightly.

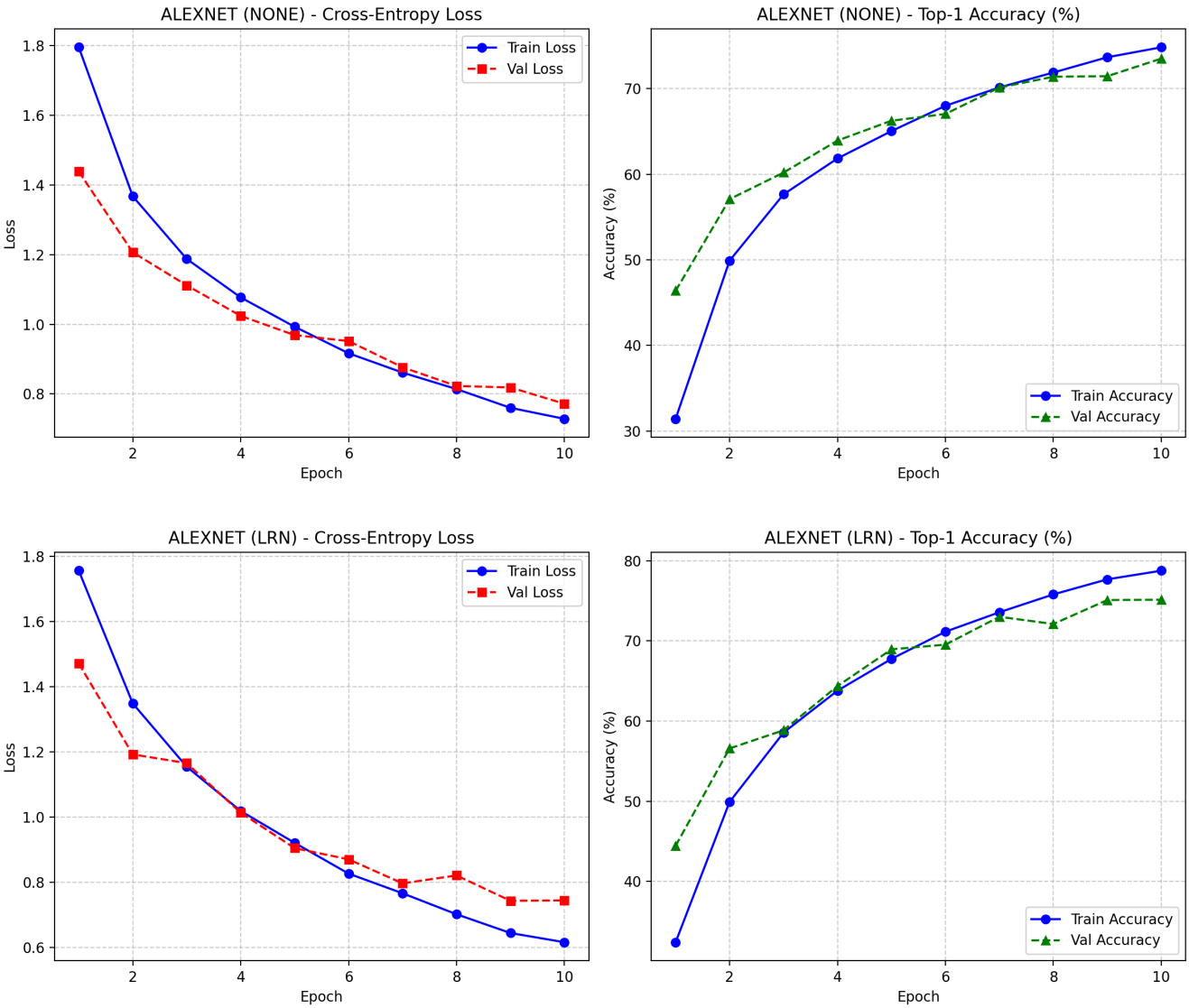
LRN added 1.90 s per epoch (+8.1%) and raised inference latency from 1.88 to 2.07 ms. BatchNorm added only 0.51 s per epoch (+2.2%). The profiler reports the same 437.17 MFLOPs for all three because it counts only convolution and linear layers. The true extra arithmetic is tiny: BatchNorm adds about 4 operations per activation, roughly 0.6 MFLOPs over the 158,912 convolution outputs (about 0.15% of the total). LRN costs more than its arithmetic suggests because its windowed sum and fractional power run as separate memory-bound GPU kernels.

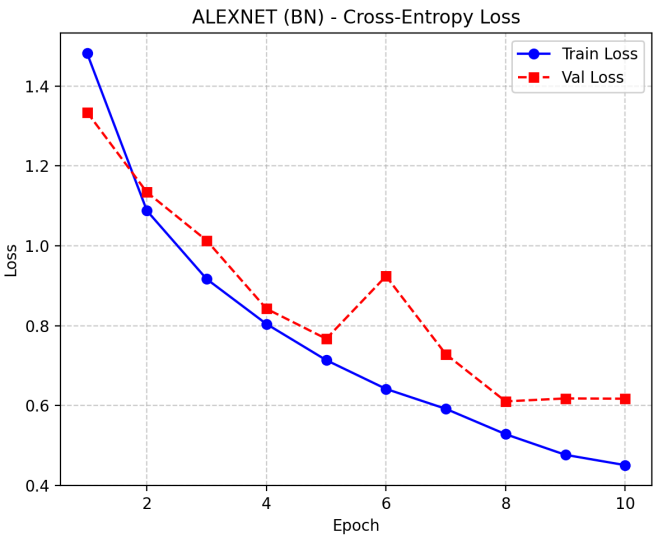
BatchNorm's lower inference latency (1.47 ms) is not a real speed-up from normalization. In evaluation mode it is a per-channel multiply-add using stored statistics, and at batch size 1 latency is dominated by kernel-launch overhead and cuDNN's kernel choice. Overall, BatchNorm's +6.4 points of accuracy for about 2% more training time is a clear win; LRN costs more and returns less.

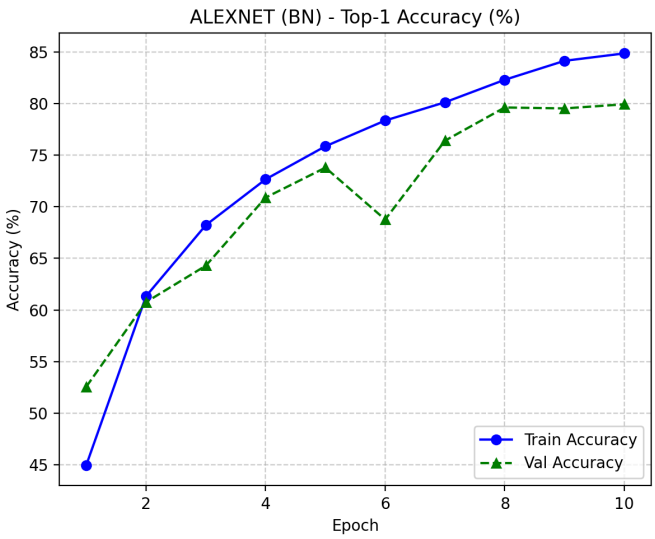
Which variant trained most stably?

All three training-loss curves fell every epoch, but BatchNorm's fell fastest and was lowest at every epoch. Its validation curve had one dip, at epoch 6 (73.79% → 68.76%, validation loss 0.77 → 0.92), before recovering to 79.62% at epoch 8. Evaluation uses running averages of the batch statistics, which lag behind the weights while they are still changing quickly. The unnormalized and LRN validation curves were smoother but stayed lower throughout, so BatchNorm gave the most stable and effective gradient progression.

Training curves





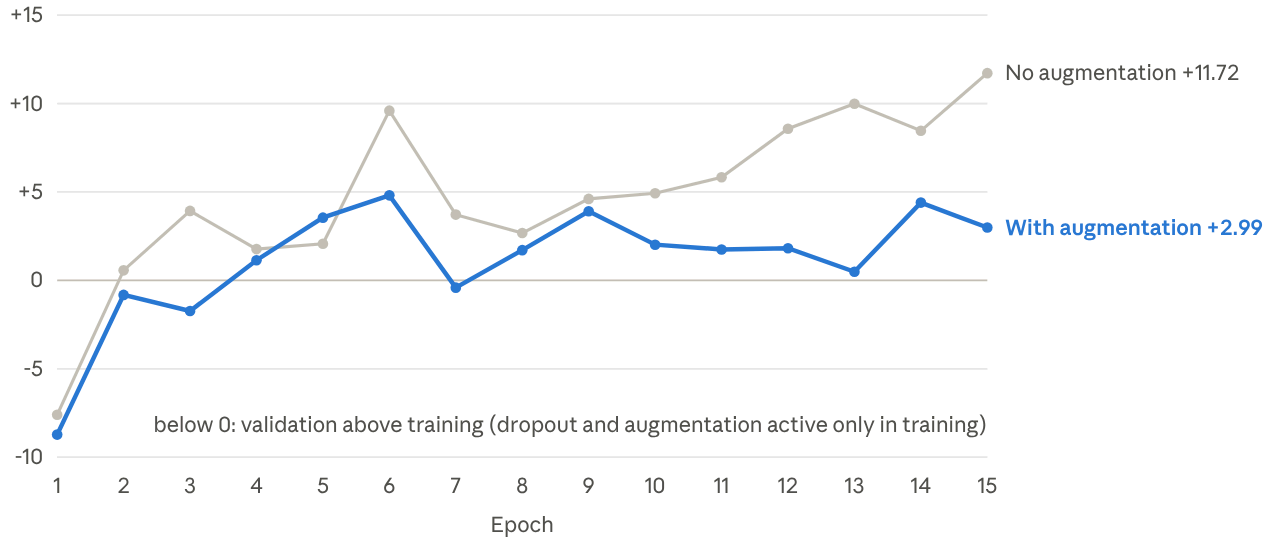


5. Data Augmentation and the Generalization Gap

The augmentation pipeline (RandomCrop 64 with 4-pixel reflect padding, RandomHorizontalFlip p = 0.5, ColorJitter 0.2 for brightness, contrast and saturation) cut the final generalization gap from 11.72 to 2.99 points and raised best validation accuracy from 81.18% to 82.83%. Both runs used AlexNet-64 with BatchNorm and Dropout for 15 epochs; validation images were only resized and normalized.

Augmentation holds the gap at 3.0 points; without it the gap widens to 11.7

AlexNet-64 + BatchNorm on CIFAR-10, generalization gap = train accuracy − validation accuracy (points), by epoch



train.py logs: alexnet_no_aug.txt, alexnet_with_aug.txt · 15 epochs each, Tesla T4

Run	Final train acc (%)	Final val acc (%)	Final gap Δ (pts)	Best val acc (%)	Best val Macro F1 (%)	Final val loss	Mean epoch time (s)
No augmentation	90.31	78.59	11.72	81.18 (epoch 14)	81.08	0.710	23.64
With augmentation	84.64	81.65	2.99	82.83 (epoch 13)	82.90	0.522	50.40

Δ = training accuracy − validation accuracy at epoch 15. Measured at each run's best epoch instead, the gap is 8.46 points without augmentation (89.64 − 81.18) and 0.49 with it (83.32 − 82.83).

Did augmentation reduce the generalization gap? Yes.

The final gap fell by 8.73 points, from 11.72 to 2.99. Without augmentation, training accuracy kept climbing to 90.31% while validation accuracy stalled near 79–81% after epoch 8. Validation loss bottomed at 0.593 in epoch 11 and then rose to 0.710, the usual sign that the network has started memorizing the training set. With augmentation, validation loss was still falling at the end (0.518 at epoch 13, 0.522 at epoch 15), and training accuracy stayed within a few points of validation accuracy.

Did augmentation improve validation accuracy and Macro F1? Yes.

Best validation accuracy rose by 1.65 points (81.18% → 82.83%) and Macro F1 by 1.82 points (81.08% → 82.90%). Each training image is now a slightly different crop, mirror and colour balance every time it is seen, while its label stays the same. The network can no longer lower its loss by memorizing exact pixel positions or a particular brightness, so its filters must respond to features that survive those changes: object shape, part layout and texture. That is the same invariance it needs for unseen validation images.

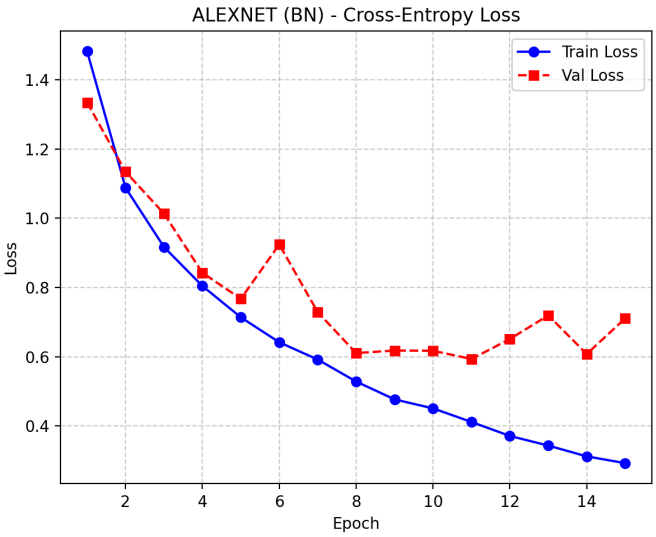
The gain in accuracy is smaller than the drop in the gap, and the augmented run had not finished improving: training accuracy was 84.64% and still rising. A longer schedule would likely widen the accuracy advantage, because augmented data takes more epochs to fit.

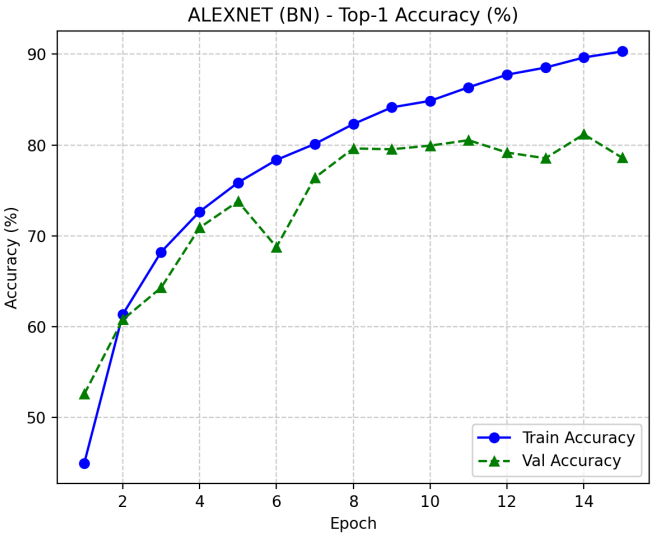
Cost

Augmentation more than doubled the epoch time (23.64 s → 50.40 s). The transforms run on PIL images in the DataLoader's CPU workers, not on the GPU, and the Colab instance has only 2 CPU cores, so the GPU waited on data loading. Running the augmentations on

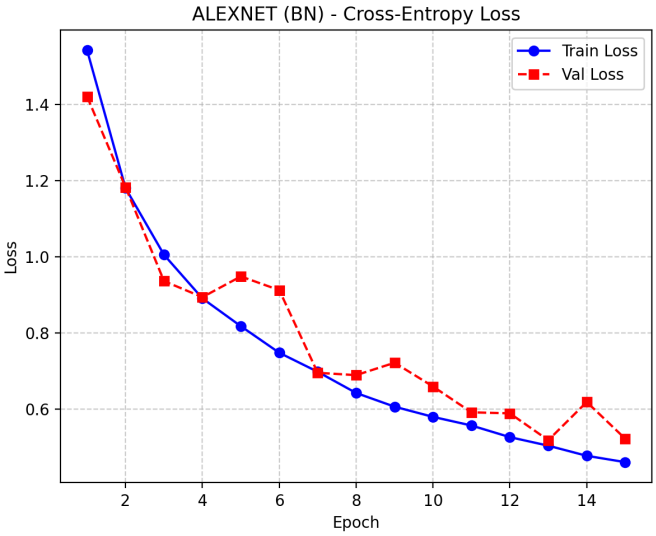
GPU tensors, or using more DataLoader workers, would remove most of this overhead. Inference cost is unchanged because augmentation is used only during training.

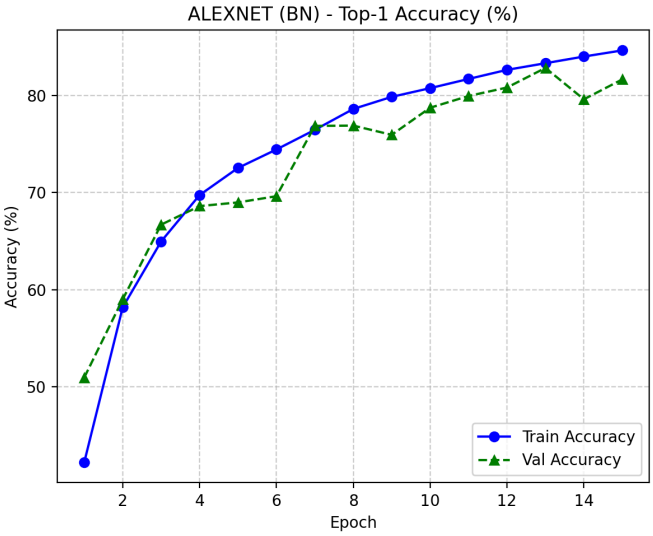
Training curves





The validation loss bottoms at epoch 11 and climbs while training loss keeps falling, the overfitting pattern described above.





With augmentation, the training and validation curves stay close together for all 15 epochs, and validation loss is still trending down at the end (0.52), so the model had not yet finished learning.

6. Dual-Dataset Benchmark: Tiny ImageNet-200

AlexNet-64 with BatchNorm reached 30.15% accuracy and 28.25% Macro F1 on the held-out Tiny ImageNet validation set after 10 epochs. That is about 60 times the 0.5% random-guess rate, but 49.8 points below the same network on CIFAR-10.

Correcting the validation split

The provided `utils/dataset.py` built the Tiny ImageNet validation loader from the `train/` folder, so the first run "validated" on the same 100,000 images it trained on (782 batches). Tiny ImageNet's official `val/` folder stores its 10,000 images in one directory with a label file, which `ImageFolder` cannot read directly. I added `prepare_tiny_imagenet_val()`, which uses `val_annotations.txt` to sort the images into one folder per class, and pointed the validation loader at `val/` (79 batches). Both folders use the same 200 WordNet IDs, so class indices match between training and validation.

Run	Evaluated on	Val acc (%)	Val Macro F1 (%)
Provided loader	100,000 training images	36.81	35.08
Corrected loader	10,000 held-out validation images	30.15	28.25

The training curves of the two runs are identical to every printed decimal (same seed, same training data), so only the evaluation changed. The 6.66-point difference is the network scoring images it has already seen, evaluated without dropout. All results below use the corrected held-out split.

Epoch	Train loss	Val loss	Train acc (%)	Val acc (%)	Val Macro F1 (%)
1	4.772	4.373	3.92	7.13	4.29
2	4.208	3.971	9.34	12.71	10.09
3	3.921	3.810	13.36	15.32	12.58
4	3.716	3.552	16.48	19.37	16.63
5	3.550	3.734	19.13	17.26	15.38
6	3.404	3.429	21.78	22.27	20.24
7	3.266	3.238	24.16	24.99	23.16
8	3.145	3.179	26.27	26.23	24.33

Epoch	Train loss	Val loss	Train acc (%)	Val acc (%)	Val Macro F1 (%)
9	3.028	3.159	28.78	27.51	25.98
10	2.924	2.995	30.49	30.15	28.25

The output layer grows to 1,024 → 200 (205,000 parameters), giving 6,806,152 parameters in total. Each epoch took 51.7 s on average on the T4, 2.2 times the CIFAR-10 epoch, because there are twice as many training images (100,000). The first run's epochs took about 90 s because it evaluated all 100,000 training images every epoch.

Did AlexNet learn features well above chance? Yes.

Random guessing over 200 balanced classes gives 0.5% accuracy, and an untrained network's loss would be ln 200 ≈ 5.30. The model reached 7.13% after one epoch and 30.15% after ten, with validation loss down to 3.00. Macro F1 (28.25%) sits only 1.9 points below accuracy, so the gain is spread across most classes rather than coming from a few easy ones.

How does it compare with CIFAR-10, and why is Tiny ImageNet harder?

Dataset	Classes	Training images per class	Native resolution	Val acc (%)	Val Macro F1 (%)
CIFAR-10	10	5,000	32×32, upscaled to 64×64	79.93	80.06
Tiny ImageNet-200	200	500	64×64	30.15	28.25

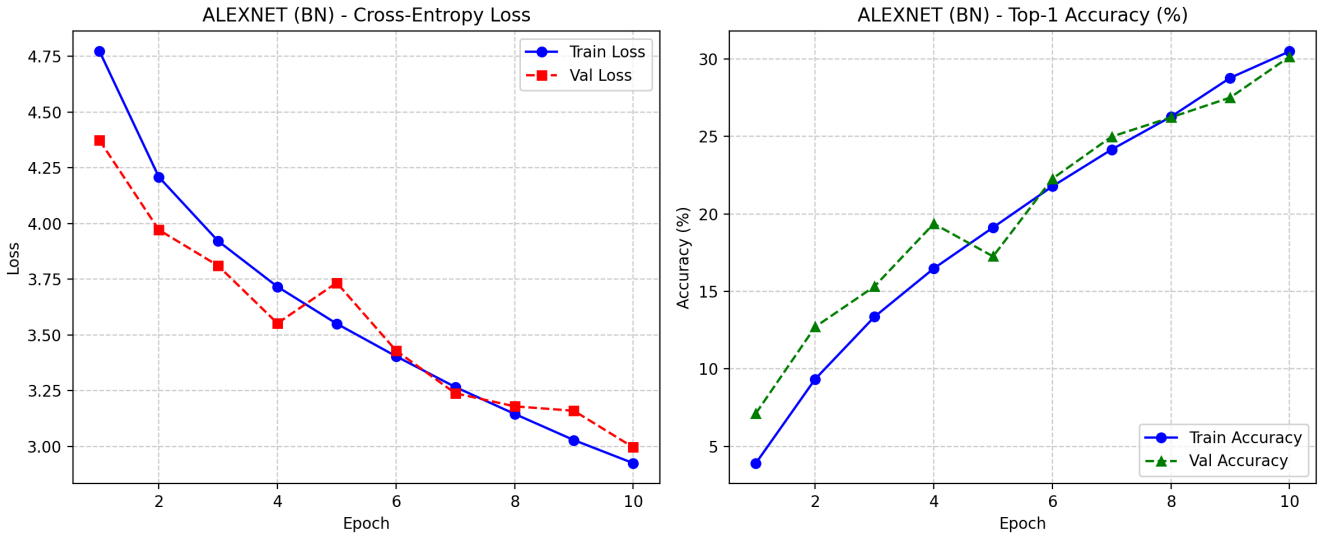
- **20 times more classes.** The chance level falls from 10% to 0.5%, and the classifier must separate 200 decision regions instead of 10.
- **10 times fewer examples per class.** With 500 images per class, each class's appearance varies far more than its training set can cover.
- **Fine-grained categories.** Tiny ImageNet includes many visually similar classes (several dog breeds, different insects, similar vehicles), while CIFAR-10's 10 classes are broad and mostly distinct.
- **Harder images.** Tiny ImageNet images are downsampled ImageNet photos with clutter, partial views and off-centre objects. CIFAR-10 objects are usually centred, and upscaling them from 32×32 adds no new detail.

Did validation loss diverge or plateau earlier than on CIFAR-10? No, it kept falling.

Validation loss fell in 9 of 10 epochs, from 4.37 to 3.00, with one bump at epoch 5 (3.55 → 3.73). Training and validation accuracy stayed within a few points of each other the whole time and ended almost equal (30.49% vs 30.15%, a gap of 0.34 points). In epochs 1–4 and 6–7 validation accuracy was even higher, because dropout is active only during training.

There is no sign of overfitting: the network is underfitting, and more epochs would keep improving it. On CIFAR-10 without augmentation, validation loss bottomed at epoch 11 and then rose while the gap grew to 11.7 points. Tiny ImageNet did not overfit earlier; it is hard enough that 10 epochs are not enough even to fit the training data (30.49% training accuracy).

Training curves



7. Discussion: Performance Across the Model Ladder

Each step up the ladder bought accuracy with more computation, and the largest single gains came from cheap design changes rather than raw size.

Step	Change	CIFAR-10 val acc change (pts)	FLOPs change	Params change
LeNet-5 → Modernized LeNet	ReLU, overlapping max pooling	+11.75 (52.98 → 64.73)	×1.28	×1.32

Step	Change	CIFAR-10 val acc change (pts)	FLOPs change	Params change
Modernized LeNet → AlexNet (no norm)	5 conv stages, 64–384 channels, dropout	+8.79 (64.73 → 73.52)	×47.6	×14.8
AlexNet (no norm) → + BatchNorm	BatchNorm after each conv	+6.41 (73.52 → 79.93)	+0.15%	+2,560
+ BatchNorm → + Augmentation	Crop, flip, colour jitter	+1.65 (81.18 → 82.83, 15 epochs each)	none	none

- **LeNet to Modernized LeNet.** ReLU does not saturate, so gradients reach the first layer at full strength instead of being shrunk by Tanh's flat tails. Max pooling keeps the strongest edge response in each window instead of averaging it away. Together these gave the largest gain for the smallest cost. The LeNet runs were 5 epochs on a CPU, so their accuracies are lower than they would be with 10 epochs.
- **Modernized LeNet to AlexNet.** Depth and width let the network build a hierarchy, from edges to parts to objects, and grow its receptive field from 19 to 97 pixels so the final features see the whole image. The price is 48 times the arithmetic.
- **BatchNorm.** The best return per unit of cost in the study: +6.4 points for about 0.15% more arithmetic and 2% more training time.
- **Augmentation.** No inference cost at all, a large drop in overfitting, and a smaller but real accuracy gain that was still growing at 15 epochs.

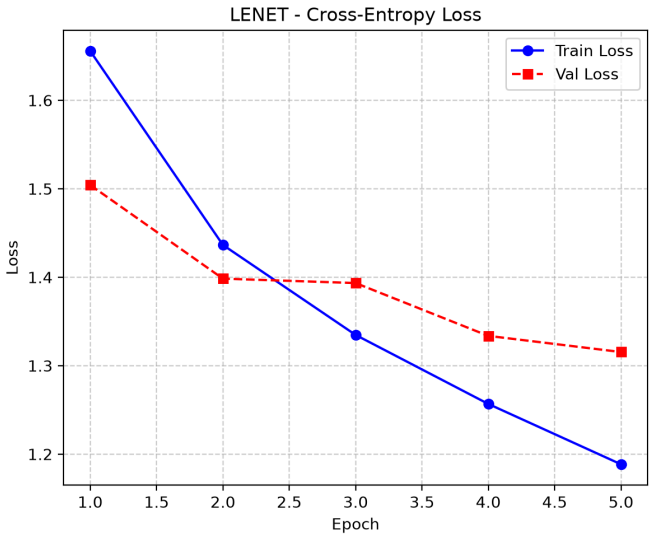
The PyTorch Zoo AlexNet

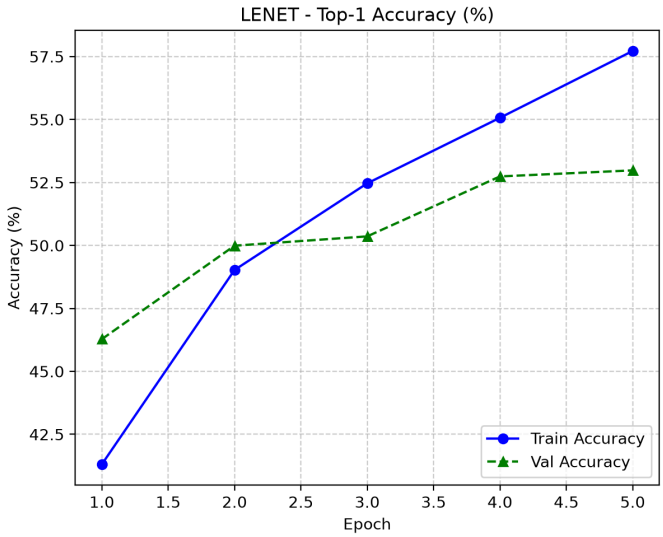
The Zoo AlexNet has 8.6 times the parameters of AlexNet-64 (57.04 M vs 6.61 M) but needs only 43% of its FLOPs (188 M vs 437 M). Its 11×11 stride-4 first layer was designed for 224×224 input. On a 64×64 image it shrinks the feature map to 1×1 by the last pool, so the convolutions do little work, while the 54.6 M dense-layer weights see only 256 distinct values copied 36 times by AdaptiveAvgPool2d. AlexNet-64's 5×5 stride-2 first layer keeps a 7×7 map through Conv3–Conv5, which is why it spends more FLOPs but far fewer parameters. The Zoo model was profiled but not trained, so its accuracy could not be compared.

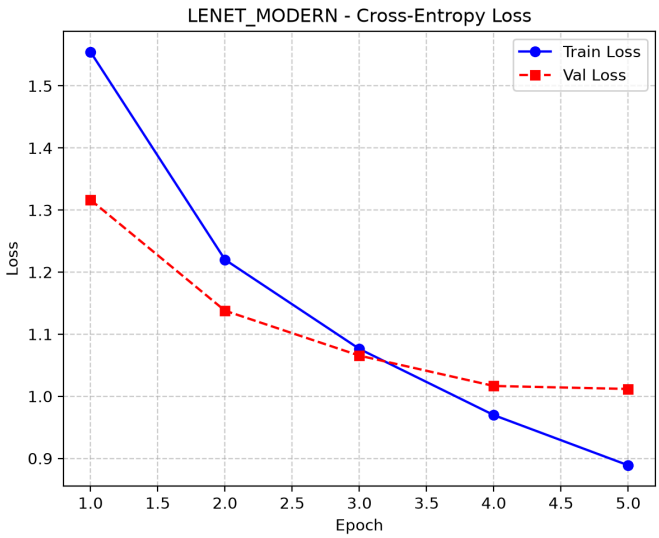
Parameters, FLOPs and latency on a GPU

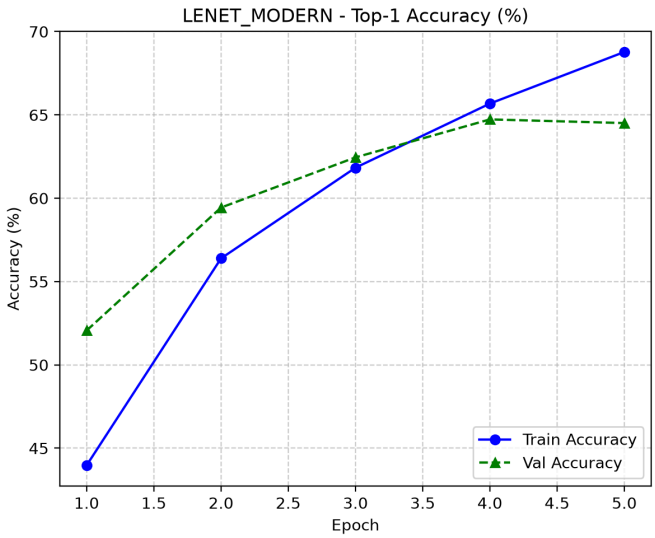
FLOPs and parameters measure different costs. In AlexNet-64, the convolutions hold 48% of the parameters but nearly all of the FLOPs; the dense layers hold 52% of the parameters but only about 7 M of the 437 M FLOPs. On the T4, inference latency barely tracks FLOPs: AlexNet-64 does 61 times the arithmetic of LeNet-5 but takes only 4.2 times as long (1.47 vs 0.35 ms). At batch size 1 a GPU is far from saturated, and latency is dominated by kernel launches and memory transfers rather than arithmetic. The same FLOP differences would matter much more on a CPU or a microcontroller, where operations run nearly in sequence; LeNet's 0.45 M parameters (1.7 MB) also fit memory budgets that AlexNet-64's 25 MB do not.

LeNet training curves (in-lab)









Modernized LeNet starts higher (52.06% vs 46.28% validation accuracy after epoch 1) and climbs faster. Its validation loss flattens by epoch 4–5 while training loss keeps falling, so an early-stopping controller would halt it soon after; base LeNet's validation loss is still descending slowly at epoch 5.

8. Conclusion

1. Modernizing LeNet with ReLU and overlapping max pooling raised CIFAR-10 accuracy from 52.98% to 64.73% for 28% more FLOPs.
2. The full AlexNet-64 (6.61 M parameters, 437 MFLOPs) reached 73.52% without normalization in 10 epochs.
3. BatchNorm was the most effective and most stable normalization, reaching 79.93% (vs 75.14% for LRN) for 2% more training time; LRN was the slowest variant and helped least.
4. Data augmentation cut the generalization gap from 11.72 to 2.99 points and raised best accuracy to 82.83%, at the cost of doubling epoch time on CPU-bound data loading.
5. On held-out Tiny ImageNet-200 images the same network reached 30.15%, 60 times chance, but still underfit after 10 epochs; more epochs and augmentation are the obvious next steps.